

Joint Catalog Selection and Pricing for Data Subscriptions

Complete Proofs

This document accompanies *Joint Catalog Selection and Pricing for Data Subscriptions*. Notation, assumptions, and all result and equation references follow the main paper, except for the supplementary operation-count bound (S1) introduced in the proof of Theorem 4.

Proof of Lemma 1 (Demand normalization)

Since $w_i > 0$, every chosen contract has positive demand. Removing zero-demand contracts preserves each type's canonical choice, leaves demands and service costs unchanged, and increases profit by the sum of the removed contracts' nonnegative activation costs. The number of contracts does not increase. $\square$

Proof of Theorem 1 (Persistent hull tree representation)

Represent the empty menu by the empty tree. For a nonempty demand-normalized menu, insert contracts in increasing index order while maintaining a stack and a tree, both initially containing only the root. The stack records the current envelope, while the tree retains the branches formed so far. After inserting all newly available contracts of each group, read its envelope from the stack. When inserting u , if the stack top x and its predecessor a satisfy

$$\frac{p_x - p_a}{q_x - q_a} \geq \frac{p_u - p_x}{q_u - q_x},$$

pop x and continue comparing until the condition fails or only the root remains. Attach u to the remaining stack top, record its slope, and push it onto the stack. Popping removes nodes only from the current stack, preserving the original nodes and incoming edges in the tree.

When popped, x lies on or above the segment joining a and u , and its utility is at most the linear interpolation of the endpoint utilities at quality q_x . When both endpoints are available, if x attains the maximum utility, both endpoints also attain that value, and the higher-quality option u is preferred. This comparison does not depend on the sign of the segment's slope. Nested permissions keep both endpoints available in subsequent groups, so x is never chosen thereafter. Demand normalization therefore ensures that every node remains on the stack through the end of its birth group, since otherwise it would have no demand in any group.

Stack slopes strictly increase, and the first edge has a nonnegative slope because prices are nonnegative. Thus, the recorded slopes are nonnegative and increase along ancestor paths. New nodes attach only to the stack top, so insertion order is preorder. If a later child y of a replaces an earlier child x , the pop condition gives $s_x \geq (p_y - p_x)/(q_y - q_x)$, while s_y is their average weighted by quality differences, so $s_y \leq s_x$. If $v(u) < P$, node u leaves the stack when $v(u)$ is inserted; otherwise, it remains through the last group. Its active range is therefore exactly the nonempty J_u . All five conditions hold, each group's stack gives its envelope, and (5) recovers the original prices.

Conversely, consider an admissible tree. Equation (5) recovers nonnegative prices from positive quality increments and nonnegative slopes. Fix access group b . The visible nodes covered by its permissions form a prefix of the nonroot preorder sequence, and an empty prefix allows only exit. Otherwise, let ℓ be the last visible node and regard P as invisible. By the definition of J_u , the incoming edge of u is active exactly when u is visible but $v(u)$ is not, meaning that ℓ lies in the subtree of u . Thus, the active edges form the path from the root to ℓ .

For a visible node z outside this path, let a be the lowest common ancestor of z and ℓ , and let x and y be its children toward the two nodes, respectively. Since ℓ is the last visible node, the branch of x precedes that of y , so preorder indexing and the quality order give $q_a < q_z < q_y$. Every edge slope on the path from a to z is at least s_x , and the sibling-slope condition gives $s_x \geq s_y$. Hence,

$$p_z - p_a \geq s_x(q_z - q_a) \geq s_y(q_z - q_a).$$

Together with $p_y - p_a = s_y(q_y - q_a)$, this places z on or above the segment joining points a and y , both available to this group. By the preceding utility comparison and the highest-quality tie rule, z is not chosen, so the active path represents the lower convex hull with collinearity allowed. Each edge on the path has utility increment $(\theta - s_u)(q_u - q_a)$. Thus, weakly increasing slopes and the highest-quality tie rule cause customers with positive valuations to traverse all edges satisfying $s_u \leq \theta$ and choose the last node reached. Types with zero valuations still exit as specified in the model. $\square$

Proof of Proposition 1 (Exact edge profit decomposition)

Type $i \in I_+$ traverses edge (a, u) exactly when $b(i) \in J_u$ and $\theta_i \geq s_u$. Summing $s_u(q_u - q_a) - (c_u - c_a)$ along its path cancels the intermediate terms and gives the unit service margin $p_{x_i} - c_{x_i}$ of the contract at its endpoint, or zero upon exit. Multiply by w_i , sum over types, and interchange the finite sums to obtain the term weighted by tail demand in (7). Every offered node has exactly one incoming edge; subtracting h_u once on that edge yields total profit. This decomposition allows negative service-cost differences and includes activation costs for zero-demand nodes. $\square$

Proof of Theorem 2 (Finite-threshold reduction and attainment)

Normalize the menu by Lemma 1. Represent an empty catalog by the empty tree. Otherwise, construct a PHT by Theorem 1. Every nonroot node u is chosen by some type $i \in I_+$, so $s_u \leq \theta_i$ and $\theta_i \in \mathcal{T}$. Every slope s in the tree can therefore be rounded up to

$$R(s) = \min\{t \in \mathcal{T} : t \geq s\}.$$

Keep the tree structure and simultaneously replace each edge slope by $R(s_u)$. Since R is nondecreasing, it preserves the weak slope inequalities along paths and between siblings. The other admissibility conditions remain unchanged. For any $i \in I_+$, $\theta_i \in \mathcal{T}$ gives

$$\theta_i \geq s_u \iff \theta_i \geq R(s_u).$$

Thus, each type traverses the same edges and reaches the same endpoint. Ties involving collinearity are still handled by the canonical rule in Theorem 1, and all other types continue to exit. Equation (5) expresses prices as sums of path slopes multiplied by positive quality increments, so prices do not decrease after rounding. Demands, costs, and the number of contracts remain unchanged, and profit does not decrease.

Admissible trees with at most $\min\{K, P\}$ nonroot nodes and edge slopes from $\mathcal{T}$ form a nonempty finite family. Each tree recovers a capacity-feasible menu, and every feasible menu corresponds to a tree in this family with no lower profit. Thus, the family's maximum profit equals the original problem's supremum and is attained by the menu recovered from a maximizing tree. $\square$

Proof of Lemma 2 (Exact fragment optimization)

Set the empty-forest boundary values, then evaluate in increasing order of k , using the order $S \rightarrow Q \rightarrow F$ for each fixed k . States S depend on forests of budget $k - 1$, Q depends on a subtree of the same budget or two parts of smaller budgets, and F depends on Q of the same budget. This order respects every dependency, so the recurrences terminate.

Removing or adding subtree root u establishes a correspondence in both directions between S and the forest in Equation (8). Other nodes retain their right boundaries. The forest's slope lower bound and the state's active-interval condition ensure a feasible attachment, with scores differing by the root contribution and node counts by 1. Every nonempty forest has a unique first child, so Equation (9) covers exactly the corresponding Q families.

For a Q forest, the first-edge slope t_x and the presence of a next sibling determine the two cases in Equation (10). When a next sibling y exists, preorder indices separate the first subtree and remaining forest

into $[u, y)$ and $[y, v)$, respectively. Node y is the first subtree's right boundary, validity of that subtree's incoming root edge gives $\beta(y) > \beta(u)$, and the sibling-slope relationship places the right forest's first edge in $[t_r, t_x]$. Conversely, joining the two parts preserves disjoint indices, sums their budgets to k , and uses the slope bounds to ensure the new sibling relationship. Every node retains its right boundary and edge contribution. Each node has one incoming edge and incurs its activation cost once, so scores are additive. Together with the evaluation order, this establishes the exactness of all three recurrences. $\square$

Proof of Theorem 3 (Exact optimization of continuous-price menus)

By Lemma 2, tree price reconstruction, and profit decomposition, every feasible root state realizes a public menu with the same score. Thus, the right-hand side of Equation (11) does not exceed the optimal profit under capacity k . Conversely, Theorem 2 converts any continuous-price menu into a finite-threshold tree with no more contracts and no lower profit. Lemma 2 covers it in some I_j with $j \leq \min\{k, P\}$, giving the reverse inequality. The finite tree family contains the empty tree, so the maximum exists. Store one maximizing transition per state and trace the finite dependency relation to recover distinct nodes and their slopes. Recover prices along the tree starting at $p_{-1} = 0$. The tree representation and edge-profit identity guarantee the resulting canonical choices and profit. $\square$

Proof of Corollary 1 (Repricing a fixed catalog)

The empty-catalog case is immediate. Retain a nonempty A in the original order; its intersection with each access prefix remains a prefix. Any fixed-catalog pricing scheme is feasible for the auxiliary problem, so its service margin is at most V_A . Conversely, take the auxiliary problem's optimal subset $A' \subseteq A$ and price each omitted contract m strictly above all input valuations $\theta_i q_m$. The restored contracts have negative utility, preserving the original choices and service margin. Subtracting the activation costs of all of A attains the stated value, including when A' is empty. $\square$

Proof of Theorem 4 (Polynomial complexity)

Count each rational arithmetic operation or index operation as one scalar operation. The numbers of states S, F, and Q are $O(P^3TK)$, $O(P^2TK)$, and $O(P^3T^2K)$, respectively, with another $O(P^3T)$ edge-value cache entries. Each Q state considers at most T first-edge thresholds, P next siblings, and K budget splits, giving total transition cost $O(P^4T^3K^2)$, which also covers the transitions of S and F.

Including input processing, birth-group lookup, tail-demand computation, menu reconstruction, and customer-choice computation, define the operation-count bound

$$\begin{aligned} \mathcal{A} = & M + N + BN + TN + BT \\ & + (B + T) \log(M + N + 2) \\ & + P^4T^3K(K + \log(B + 1)) \\ & + (N + B)K + K \log(K + 1). \end{aligned} \tag{S1}$$

Let L bound the binary numerator and denominator lengths of input rationals, and let $\mathcal{J}$ be the complete input encoding length. The thresholds include positive valuation coefficients from all input types, including those with empty permissions, so $T \leq N + 1$ and $B \leq \min\{M, N\}$.

The counts above give the main costs of states and transitions. Binary searches for birth groups cost $O(P^4T^3K \log(B + 1))$. Type grouping, tail demands, and their prefix sums over group indices require $O(BN + TN + BT)$ operations, allowing each edge with known birth groups to be scored with a constant number of rational operations. Input checks and sorting are included in the first two lines of Equation (S1). Root budgets share the state tables, and prefix maxima give the capacity frontier. One reconstruction visits $O(K)$ pointers. Sorting nodes, constructing group paths, and computing choices for all original types require $O(K \log(K + 1) + (B + N)K)$ operations, giving $O(\mathcal{A})$ in total.

In a random-access model that charges bit costs, tail demand sums at most N values w_i , whose denominator product can serve as a common denominator. A single-edge contribution requires a constant

number of additional rational operations, so the numerators and denominators of tail demands and single-edge contributions can each be represented with

$$\Lambda = O((N+1)L + \log(N+1) + 1)$$

bits. Every feasible dynamic-programming candidate sums at most K edge contributions. State S adds one edge, Q joins disjoint node sets, and maximization only selects one candidate. The numerator and denominator bit lengths of a candidate value are therefore bounded by

$$D = O((K+1)\Lambda + L + \log(K+1)).$$

Temporary cross-products and comparisons increase bit lengths by only a constant factor. The common denominator of reconstructed prices divides the product of the selected quality-coefficient and threshold denominators, which has at most $2KL$ bits. Together with $p_m \leq t_{T-1}q_m$, this also gives polynomial numerator lengths. Including the denominators of w_i and costs keeps the bit lengths in demand and profit computation within $O(D)$.

Table indices require $O(\log(M+N+2))$ bits. With elementary integer arithmetic and Euclidean reduction, each scalar operation has bit cost at most $O((D + \log(M+N+2))^3)$, giving total bit time

$$O(\mathcal{J} + \mathcal{A}(D + \log(M+N+2))^3).$$

Each state stores only one rational value and a constant number of indices. Together with BT tail-demand prefix-sum entries and the reconstructed output, the space requirement is also polynomial. $\square$

Proof of Proposition 2 (Tie rules and optimal value)

First fix a catalog A . The empty catalog has zero profit. For nonempty A , let $U = 1 + \max_{i,m \in A} \theta_i q_m$, taking $U = 1$ when there are no customer types. A contract priced above U is never chosen, and reducing its price to U leaves its utility negative, preserving the assignment and profit. Prices can therefore be restricted to $[0, U]^{|A|}$. There are finitely many assignments. For each feasible assignment, the prices satisfying utility maximization form a closed bounded polytope, and profit is linear in prices. Hence an optimal price–assignment pair (p, x) exists.

Choose distinct positive constants d_m for the contracts, and set $p_{-1} = c_{-1} = d_{-1} = 0$ for exit. For $0 < \delta < 1$, perturb the public prices to

$$\hat{p}_m(\delta) = (1 - \delta)p_m + \delta c_m + \delta^2 d_m. \quad (12)$$

Contract prices are strictly positive, and the change in an available contract's utility is $\delta(p_m - c_m) - \delta^2 d_m$. Since types and options are finite, a single sufficiently small δ preserves the signs of all originally nonzero utility differences, lets first-order margin differences resolve original utility ties, and resolves the remaining ties in favor of smaller d_m . Each type thus has a unique utility-maximizing option, selected successively by original utility, original margin, and smaller d_m . Zero-valuation types strictly exit. Zero-price contracts originally tied with exit have margin $-c_m \leq 0$, consistent with this ordering.

For each type, the choice under the perturbation has a margin at the original prices no smaller than that of the original assignment. Summing with weights w_i and subtracting the fixed activation cost $\sum_{m \in A} h_m$ shows that the profit limit as $\delta \rightarrow 0$ is no smaller than the profit of (p, x) . If an option with zero original utility and margin is replaced by exit, its contribution still converges to zero. All tie rules agree on strict choices, and their choices are weak-IC assignments whose profit cannot exceed the weak-IC optimum. Thus all rules have the same supremum on A . There are finitely many catalogs within the capacity limit, so this relation extends to joint catalog and price optimization. Theorem 2 ensures attainment of the joint optimum under the canonical rule, and Corollary 1 ensures attainment for a fixed catalog. $\square$